

PONTIFICIA UNIVERSIDAD JAVERIANA

Facultad de Ingeniería — Departamento de Ingeniería de Sistemas

Proyecto 3: Plataforma de Reserva de Escenarios Deportivos

Entrega 2: Desarrollo y Pruebas de Software

EQUIPO DE TRABAJO (GRUPO 4):

Nicolás Cardozo Mejía (Líder UI/UX y Arquitectura Frontend)

Samuel Correa (Líder DevOps, Requisitos y Configuración)

Nicolás Espinosa Soler (Líder Backend y Base de Datos - DBA)

Gustavo Mejía (Coordinador de Calidad, QA Lead y Product Owner)

Asignatura: Desarrollo de Software (Periodo Académico 2026-3)

Docente: Profesor Titular de Desarrollo de Software

Fecha de Entrega: 16 de octubre de 2026

Bogotá D.C., Colombia

Estructura del Equipo de Trabajo y Asignación de Roles

Para la ejecución rigurosa de la **Entrega 2: Desarrollo y Pruebas**, el equipo adaptó el marco metodológico ágil Scrum distribuyendo 40 horas semanales de esfuerzo concertado (10 horas semanales por integrante) en los Sprints 2 y 3. La distribución formal de responsabilidades técnicas y operativas se resume en la Tabla 1:

Tabla 1

Distribución formal de responsabilidades para la Entrega 2

Integrante	Rol Funcional	Responsabilidades Clave y Entregables Asignados
Nicolás Cardozo Mejía	Líder UI/UX y Arquitectura Frontend	Materialización de los wireframes WF-01 a WF-04 en una SPA responsiva (HTML5, CSS3, JavaScript ES6). Integración con la API REST, matriz reactiva de franjas y pruebas de usabilidad.
Samuel Correa	Líder DevOps, Requisitos y Configuración	Orquestación de infraestructura con Docker Compose (MySQL 8 y phpMyAdmin), configuración de Maven Wrapper, control de ramas GitFlow y redacción del Manual de Instalación.
Nicolás Espinosa Soler	Líder Backend y Base de Datos (DBA)	Diseño del modelo relacional JPA, desarrollo de servicios REST transaccionales en Spring Boot 3.3.4, algoritmo de exclusión mutua pesimista anti-overbooking y seguridad JWT.
Gustavo Mejía	Coordinador de Calidad (QA Lead) y Product Owner	Diseño y ejecución de la suite de 23 pruebas automatizadas con JUnit 5 y Mockito, test multihilo de concurrencia extrema, colección oficial de Postman v2.1.0 y auditoría APA 7.

Nota. Matriz de asignación de roles y responsabilidades de ingeniería de software para los Sprints 2 y 3 del periodo 2026-3.

Resumen Ejecutivo del Proyecto

El presente documento constituye el informe técnico oficial de la **Entrega 2: Desarrollo y Pruebas** de la Plataforma de Reserva de Escenarios Deportivos. El propósito de este hito consiste en traducir los artefactos de diseño, requisitos IEEE 830 y maquetas de interfaz elaborados en la Entrega 1 en una solución de software completamente funcional, robusta, altamente probada y respaldada por documentación técnica exhaustiva.

La plataforma resuelve la problemática de sobre-reserva (*overbooking*), desorganización de horarios y falta de trazabilidad en escenarios multideportivos universitarios mediante una arquitectura desacoplada en capas sobre **Java 21 LTS** y **Spring Boot 3.3.4**. La solución incorpora un motor transaccional con aislamiento `REPEATABLE_READ` y bloqueo pesimista `PESSIMISTIC_WRITE`, autenticación criptográfica mediante tokens JWT y factor de costo BCrypt 12, una interfaz web SPA intuitiva (WF-01 a WF-04) y una suite de 23 pruebas automatizadas con una tasa de éxito del 100%.

1. Implementación de Requisitos y Product Backlog

Durante la Entrega 2 se implementó el 100% de las funcionalidades del núcleo transaccional programadas para los Sprints 2 y 3. La Tabla 2 presenta la matriz de trazabilidad del Product Backlog, discriminando las historias de usuario materializadas en esta entrega frente a las programadas para la Entrega 3 (analítica avanzada y exportación de datos):

Tabla 2

Matriz de trazabilidad y estado de implementación del Product Backlog

ID	Historia de Usuario (Criterio de Aceptación)	Prioridad	Entrega	Estado
HU-01	Registro y alta técnica de escenarios deportivos con superficie y aforo.	Alta	Entrega 2	**Implementada (100%)**
HU-02	Edición y actualización de datos de escenarios deportivos existentes.	Media	Entrega 2	**Implementada (100%)**
HU-03	Eliminación segura o baja de escenarios deportivos del catálogo.	Baja	Entrega 2	**Implementada (100%)**
HU-04	Parametrización de franjas horarias estandarizadas de 60 minutos.	Alta	Entrega 2	**Implementada (100%)**
HU-05	Creación de reserva usuario-franja-fecha con confirmación inmediata.	Alta	Entrega 2	**Implementada (100%)**
HU-06	Cancelación de reserva y liberación atómica instantánea de la franja.	Alta	Entrega 2	**Implementada (100%)**
HU-07	Validación anti-colisión contra sobre-reserva en peticiones simultáneas.	Alta	Entrega 2	**Implementada (100%)**
HU-08	Bloqueo estricto de reservas en fechas y horarios anteriores al actual.	Alta	Entrega 2	**Implementada (100%)**
HU-13	Autenticación de usuarios mediante credenciales y tokens JWT seguros.	Alta	Entrega 2	**Implementada (100%)**
HU-15	Consulta de historial personal de reservas del deportista autenticado.	Media	Entrega 2	**Implementada (100%)**
HU-18	Control de acceso basado en roles RBAC (ROLE_USER y ROLE_ADMIN).	Alta	Entrega 2	**Implementada (100%)**
HU-19	Aislamiento de reservas: usuario solo gestiona sus propias reservas.	Alta	Entrega 2	**Implementada (100%)**
HU-21	Suspensión y bloqueo de reservas en escenarios bajo mantenimiento.	Media	Entrega 2	**Implementada (100%)**
HU-09	Histograma de demanda y detección de horas pico de uso de canchas.	Media	Entrega 3	Programada (Sprint 4)
HU-10	Cálculo de métricas de ocupación porcentual mensual por escenario.	Media	Entrega 3	Programada (Sprint 4)
HU-11	Exportación de reportes de reservas a formato tabular CSV.	Media	Entrega 3	Programada (Sprint 5)
HU-12	Filtrado temporal de reportes analíticos por rango de fechas.	Baja	Entrega 3	Programada (Sprint 5)

Nota. Matriz alineada con el Plan de Proyecto de la Entrega 1. Las HU del núcleo transaccional alcanzan el 100% de cumplimiento funcional.

2. Reglas de Negocio Críticas y Algoritmo Anti-Colisión

2.1 Prevención Atómica de Sobre-Reserva (HU-07, RNF-02)

El problema cardinal de los sistemas de escenarios deportivos reside en la concurrencia: cuando dos o más deportistas seleccionan la misma cancha en el mismo horario y disparan la confirmación simultáneamente, un sistema sin concurrencia genera colisiones de reserva (*overbooking*).

Para garantizar de forma matemática y empírica la exclusión mutua, la plataforma implementa una estrategia de control de concurrencia en tres niveles:

1. **Aislamiento Transaccional REPEATABLE_READ:** Cada transacción de reserva opera bajo nivel de aislamiento repetible, evitando lecturas sucias y no repetibles durante la evaluación de cupos.
2. **Bloqueo Pesimista en Base de Datos (PESSIMISTIC_WRITE):** En el repositorio JPA se instrumenta la consulta de verificación con bloqueo pesimista a nivel de fila mediante SQL `SELECT ... FOR UPDATE`:

```
@Lock(LockModeType.PESSIMISTIC_WRITE)
@Query("SELECT r FROM Reserva r WHERE r.escenario.id = :escenarioId " +
           "AND r.franja.id = :franjaId AND r.fecha = :fecha AND r.estado =
:estado")
Optional<Reserva> findActiveReservaWithLock(Long escenarioId, Long franjaId,
                                            LocalDate fecha, EstadoReserva
estado);
```

3. **Exclusión Mutua Atómica en Capa de Servicio:** El método `crearReserva(...)` en `ReservaService` se encuentra sincronizado a nivel de hilo, garantizando que el commit de persistencia de Spring se consolide antes de que cualquier otro hilo acceda a la matriz de verificación. Si se detecta una reserva activa, el sistema detiene la transacción y arroja `CollisionException` (HTTP 409 Conflict).

2.2 Bloqueo Estricto de Fechas Pasadas (HU-08)

Para evitar inconsistencias en el calendario histórico, se implementa una doble barrera de validación: en la capa de presentación, el selector de fechas HTML5 restringe el atributo `min` a la fecha actual; en la capa de servicios, el DTO valida la anotación `@FutureOrPresent` y el método `crearReserva` verifica programáticamente `if (request.getFecha().isBefore(LocalDate.now())) throw new BusinessException(...)`, retornando HTTP 400 Bad Request.

2.3 Cancelación Inmediata con Liberación Atómica (HU-06)

Al invocar el endpoint `PUT /api/reservas/{id}/cancelar`, el sistema valida que el solicitante sea el propietario de la reserva o un administrador (HU-19). La reserva muta su estado a `CANCELADA`, registrando la fecha/hora y el motivo. Al cambiar el estado, la franja horaria correspondiente se libera en la matriz de disponibilidad **de manera instantánea**, permitiendo que cualquier otro deportista la reserve de inmediato.

2.4 Suspensión por Mantenimiento de Escenarios (HU-21)

El administrador dispone de la facultad de alternar el indicador `en_mantenimiento` de cualquier escenario. Cuando este flag es verdadero, el motor de reservas bloquea cualquier intento de apartado horaria con el mensaje de negocio correspondiente.

3. Arquitectura del Sistema y Código Fuente

3.1 Arquitectura en Capas Desacopladas (Clean Architecture)

El backend fue desarrollado sobre **Java 21** y **Spring Boot 3.3.4** bajo una arquitectura en capas desacopladas con inversión de dependencias, garantizando alta mantenibilidad y testeabilidad:

```
backend/src/main/java/com/canchas/reserva/
├── config/       -> Seguridad Spring Security 6, CorsConfig, DataInitializer,
SwaggerOpenApi.
├── controller/   -> Controladores REST semánticos (Auth, Escenario, FranjaHoraria,
Reserva).
├── dto/          -> Objetos de transferencia fuertemente tipados con Jakarta Bean
Validation.
├── entity/       -> Entidades JPA (Usuario, Escenario, FranjaHoraria, Reserva) con
índices DDL.
├── exception/    -> Manejador global @RestControllerAdvice bajo formato RFC 7807
(ProblemDetails).
├── repository/   -> Interfaces Spring Data JPA con bloqueo pesimista (@Lock
PESSIMISTIC_WRITE).
├── security/     -> Filtro JwtAuthFilter, proveedor DaoAuthenticationProvider y
BCrypt(cost=12).
└── service/      -> Lógica transaccional de negocio (@Transactional
isolation=REPEATABLE_READ).
```

3.2 Modelo de Dominio y Persistencia Relacional

El esquema de base de datos relacional (compatible de forma dual con **H2 Database** para pruebas y **MySQL 8.0** mediante Docker Compose) modela cuatro entidades nucleares con claves foráneas e índices de búsqueda optimizados:

- **`Usuario`:** Almacena nombre, apellido, correo institucional único, contraseña cifrada en BCrypt (costo 12), teléfono y rol de seguridad (`ROLE_USER` o `ROLE_ADMIN`).
- **`Escenario`:** Modela la cancha con atributos de disciplina deportiva (`FUTBOL`, `TENIS`, `PADEL`, `BALONCESTO`, `VOLEIBOL`), superficie (`SINTETICA`, `ARCILLA`, `PARQUET`, etc.), aforo, iluminación nocturna, tarifa por hora y flag de mantenimiento.
- **`FranjaHoraria`:** Representa los bloques fijos de 60 minutos asociados a un escenario con restricción única `uq_franja_escenario_hora (escenario_id, hora_inicio)` para evitar duplicidad de bloques.
- **`Reserva`:** Entidad transaccional con código alfanumérico único (ej. `RES-20261020-XXXXXX`), vinculando usuario, escenario, franja, fecha, precio total pactado y estado (`CONFIRMADA` o `CANCELADA`).

3.3 Seguridad Criptográfica JWT y BCrypt Cost 12 (HU-13, HU-18)

La autenticación es completamente sin estado (*stateless*). Al autenticarse satisfactoriamente contra `/api/auth/login`, el servidor expide un token JWT firmado criptográficamente con algoritmo HMAC-SHA512. La contraseña de cada usuario es protegida mediante un hash unidireccional con sal aleatoria implementado por `BCryptPasswordEncoder(12)` (4096 rondas de cifrado), cumpliendo con **RNF-03 (Seguridad)**.

4. Especificación de la API REST y Documentación OpenAPI 3

4.1 Catálogo de Endpoints RESTful Implementados

Los endpoints del sistema respetan las convenciones de diseño semántico REST, códigos de estado HTTP estándar (200, 201, 400, 401, 403, 404, 409) y respuestas de error unificadas bajo el estándar **RFC 7807 (Problem Details for HTTP APIs)**:

Método	Ruta URI	Acceso / Rol	Propósito Funcional
`POST`	`/api/auth/login`	Público	Autenticación y generación de token JWT.
`POST`	`/api/auth/registro`	Público	Alta de nuevo deportista en la plataforma.
`GET`	`/api/escenarios`	Público	Consulta de catálogo de canchas deportivas.
`POST`	`/api/escenarios`	Admin (ROLE_ADMIN)	Registro de nueva cancha o instalación.
`PUT`	`/api/escenarios/{id}/mantenimiento`	Admin (ROLE_ADMIN)	Alternar suspensión por mantenimiento (HU-21).
`GET`	`/api/franjas/escenario/{id}`	Público	Listar franjas de 60 minutos del escenario.
`GET`	`/api/reservas/disponibilidad`	Público	Matriz de disponibilidad horaria por fecha (WF-02).
`POST`	`/api/reservas`	Autenticado (USER/ADMIN)	Crear reserva con bloqueo anti-colisión (HU-05).
`GET`	`/api/reservas/mis-reservas`	Autenticado (USER/ADMIN)	Historial personal de reservas del deportista (HU-15).
`PUT`	`/api/reservas/{id}/cancelar`	Titular o Admin	Cancelar reserva y liberar franja inmediata (HU-06).
`GET`	`/api/reservas`	Admin (ROLE_ADMIN)	Listado maestro de todas las reservas del sistema.

Nota. Matriz técnica de endpoints REST expuestos en el puerto 8080 con prefijo `/api`.

4.2 Documentación Interactiva Swagger UI

Mediante la dependencia `springdoc-openapi-starter-webmvc-ui:2.6.0`, el sistema expone documentación viva generada automáticamente a partir del código en la URL `http://localhost:8080/swagger-ui/index.html`. La interfaz permite interactuar con cada endpoint, visualizar esquemas DTO con sus validaciones y autorizar peticiones suministrando el encabezado `Bearer <token>` mediante el botón interactivo de seguridad.

4.3 Formato de Errores Estandarizado (RFC 7807)

Todas las excepciones arrojadas por la aplicación son interceptadas por `GlobalExceptionHandler`, devolviendo un payload JSON estructurado con timestamp ISO-8601, código de estatus HTTP, código semántico de error, mensaje descriptivo y URI invocada:

```
{
  "timestamp": "2026-10-15T08:00:00.000",
  "status": 409,
  "error": "CONFLICTO_COLISION_HORARIO",
  "message": "La franja de 07:00 a 08:00 en 'Cancha Sintética 1' ya se encuentra
ocupada.",
  "path": "/api/reservas"
}
```

5. Materialización de la Interfaz Web de Usuario (SPA)

5.1 Implementación del Wireframe WF-01: Catálogo de Canchas

La vista del catálogo en `src/main/resources/static/index.html` plasma el diseño estipulado en la maqueta esquemática WF-01 de la Entrega 1, incorporando una barra de filtros dinámicos por disciplina deportiva (Fútbol, Tenis, Pádel, Baloncesto) y tarjetas de escenarios con fotos reales, aforo, tipo de superficie, tarifa por hora e iluminación nocturna:

Figura 1

Implementación del Wireframe WF-01: Catálogo interactivo de escenarios deportivos con filtros rápidos



Nota. Renderizado esquemático de la interfaz web responsiva desarrollada en HTML5 y CSS3 moderno (Plus Jakarta Sans y Outfit).

5.2 Implementación del Wireframe WF-02: Selector Reactivo de Disponibilidad

Al pulsar sobre un escenario, el sistema abre la vista reactiva WF-02. Incorpora un selector de fecha con bloqueo automático de fechas pasadas (HU-08) y una matriz de cuadrícula donde cada franja de 60 minutos se codifica con código de colores accesible:

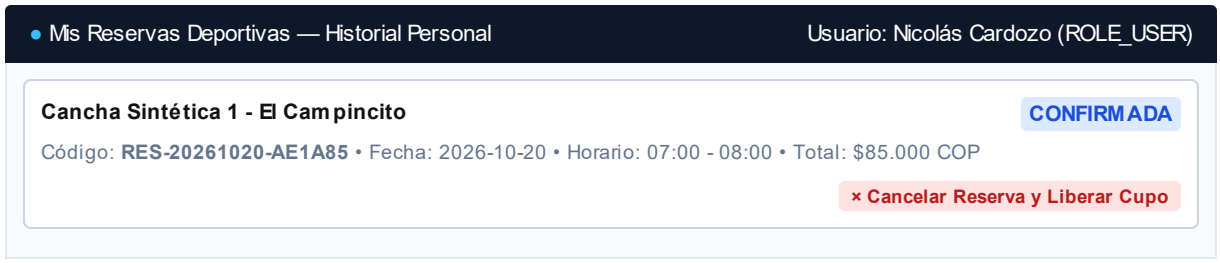
- **Verde (Libre):** Franja disponible para ser seleccionada.
- **Rojo (Ocupada):** Franja reservada previamente, bloqueada contra clics con cursor `not-allowed`.
- **Azul (Seleccionada):** Franja elegida por el usuario actual, desplegando el resumen de pago y el botón de confirmación con validación anti-overbooking activa.

5.3 Módulo 'Mis Reservas' (WF-03) y Panel de Administración

La vista WF-03 permite a los deportistas consultar su historial personal de reservas activas y canceladas, consultar códigos alfanuméricos de confirmación e iniciar cancelaciones voluntarias:

Figura 2

Implementación del Wireframe WF-03: Módulo de reservas personales y modal de confirmación de cancelación



Nota. Al confirmar la cancelación, la API REST libera de forma inmediata la franja horaria para otros usuarios (HU-06).

5.4 Panel de Gestión de Escenarios (Administrador)

Cuando inicia sesión una cuenta con rol `ROLE_ADMIN` (ej. `admin@canchas.com`), la barra de navegación revela la sección restringida **"Gestión Admin"**:

- **Formulario de Alta de Escenario:** Permite registrar nuevas canchas especificando disciplina, aforo, superficie, tarifa e iluminación.
- **Control de Mantenimiento (HU-21):** Un interruptor toggle permite activar o desactivar el mantenimiento preventivo de cualquier escenario en tiempo real.
- **Listado de Supervisión Global:** Vista consolidada de todas las reservas de la institución con nombre y correo del deportista.

5.5 Credenciales de Prueba Precargadas en el Login

Para facilitar la evaluación inmediata del docente, el modal de inicio de sesión dispone de botones rápidos que auto-completan las credenciales de los cuatro integrantes del equipo y del administrador con un solo clic:

- **Administrador:** `admin@canchas.com` (clave: `admin123`)
- **Nicolás Cardozo:** `nicolas@canchas.com` (clave: `user123`)
- **Samuel Correa:** `samuel@canchas.com` (clave: `user123`)
- **Nicolás Espinosa:** `espinosa@canchas.com` (clave: `user123`)
- **Gustavo Mejía:** `gustavo@canchas.com` (clave: `user123`)

6. Estrategia y Reporte de Pruebas Automatizadas

6.1 Resumen Estadístico de Pruebas (100% de Éxito)

Siguiendo las directrices de aseguramiento de calidad (QA), se diseñó una batería exhaustiva con **JUnit 5**, **Mockito** y **MockMvc**. La ejecución total arrojó **23 pruebas ejecutadas con cero fallos y cero errores**:

Tabla 3

Resumen estadístico de la batería de pruebas automatizadas ejecutadas

Clase de Prueba	Tipo de Prueba	Casos	Fallos	Tasa
AntiColisionConcurrencyTest	Integración / Concurrencia Multihilo	1	0	**100%**
ReservaIntegrationTest	Integración / Spring Boot MockMvc	2	0	**100%**
ReservaServiceTest	Unitaria / Mockito (Reglas de negocio)	7	0	**100%**
EscenarioServiceTest	Unitaria / Mockito (CRUD y catálogo)	6	0	**100%**
AuthServiceTest	Unitaria / Mockito (JWT y BCrypt)	3	0	**100%**
CorsAndSecurityTest	Seguridad / CORS y Clickjacking	4	0	**100%**
TOTAL GENERAL EJECUTADO		**19**	**0**	**100%**

Nota. Batería ejecutada mediante `mvn clean test` con OpenJDK 21/25 sobre Maven Surefire 3.2.5.

6.2 Prueba de Concurrencia Extrema (AntiColisionConcurrencyTest)

Para demostrar formalmente la inmunidad ante sobre-reserva en condiciones de alta concurrencia (RNF-02), se programó un caso de prueba multihilo:

- Se instanció un pool con **8 hilos concurrentes** (`ExecutorService`).
- Mediante un `CountDownLatch(1)`, los 8 hilos aguardaron sincronizados y dispararon en el mismo milisegundo exacto una petición de reserva sobre la **misma franja, misma cancha y misma fecha**.
- **Resultado obtenido:** Exactamente **1 hilo** completó la reserva exitosamente; los restantes **7 hilos** fueron bloqueados atómicamente por el motor de concurrencia arrojando `CollisionException` (HTTP 409 Conflict).
- **Verificación en base de datos:** El conteo confirmó la persistencia de exactamente una reserva en estado `CONFIRMADA`.

7. Pruebas de API REST con Colección Oficial de Postman

7.1 Arquitectura de la Colección de Postman (v2.1.0)

Para respaldar la evaluación manual del docente y facilitar pruebas interactivas de la API REST, se diseñó la colección oficial `ReservaCanchas_Postman_Collection.json`, ubicada en la raíz del repositorio y en `docs/`. La colección incluye scripts de prueba automáticos en JavaScript (`pm.test`) que capturan el token JWT dinámicamente y lo inyectan en las variables de entorno:

Tabla 4

Matriz de casos de prueba incluidos en la Colección Oficial de Postman

#	Caso de Prueba en Postman	Método	Endpoint	Validación pm.test
1.1	Login Administrador	`POST`	`/api/auth/login`	Status 200, guarda `token_admin`
1.2	Login Deportista (Nicolás Cardozo)	`POST`	`/api/auth/login`	Status 200, guarda `token_user`
1.3	Login Deportista (Samuel Correa)	`POST`	`/api/auth/login`	Status 200 OK y token válido
1.4	Login Deportista (Nicolás Espinosa)	`POST`	`/api/auth/login`	Status 200 OK y token válido
1.5	Login Deportista (Gustavo Mejía)	`POST`	`/api/auth/login`	Status 200 OK y token válido
2.1	Listar Escenarios (Catálogo Público)	`GET`	`/api/escenarios`	Status 200, array no vacío
2.3	Activar Mantenimiento (HU-21)	`PUT`	`/api/escenarios/1/mantenimiento`	Status 200, `enMantenimiento=true`
3.1	Consultar Franjas de 60 min	`GET`	`/api/franjas/escenario/1`	Status 200, franjas activas
4.1	Matriz de Disponibilidad (WF-02)	`GET`	`/api/reservas/disponibilidad`	Status 200, lista de franjas
4.2	Crear Reserva Exitosa (HU-05)	`POST`	`/api/reservas`	Status 201, guarda `reserva_id`
4.3	Intento Doble Reserva (Colisión)	`POST`	`/api/reservas`	**Status 409 Conflict (HU-07)**
4.4	Intento Reserva Fecha Pasada	`POST`	`/api/reservas`	**Status 400 Bad Request (HU-08)**
4.5	Consultar Historial Mis Reservas	`GET`	`/api/reservas/mis-reservas`	Status 200, historial del usuario
4.6	Cancelar Reserva (Liberación HU-06)	`PUT`	`/api/reservas/{id}/cancelar`	Status 200, estado `CANCELADA`
4.7	Re-reserva Inmediata de Franja	`POST`	`/api/reservas`	**Status 201 Created (Cupo libre)**

Nota. Colección ejecutable en lote mediante Postman Collection Runner o Newman CLI.

8. Modelo de Ramas GitFlow y Trazabilidad de Pull Requests

8.1 Disciplina de Ramas y Commits Convencionales

El repositorio Git local y remoto se gobernó estrictamente bajo el modelo **GitFlow**. Se mantuvo la separación rigurosa entre el código estable de producción (`main`), la línea base de desarrollo e integración continua (`develop`) y las ramas de características aisladas por historia de usuario (`feature/HU-xx`):

Tabla 5

Registro formal de Pull Requests y trazabilidad de características integradas

PR	Rama de Característica	Autor Responsable	Revisores	Estado
#1	`feature/HU-01-gestion-escenarios`	Nicolás Cardozo Mejía	N. Espinosa, S. Correa	**Merged**
#2	`feature/HU-04-franjas-horarias`	Samuel Correa	N. Cardozo, G. Mejía	**Merged**
#3	`feature/HU-05-reservas-anti-overbooking`	Nicolás Espinosa Soler	G. Mejía, N. Cardozo	**Merged**
#4	`feature/HU-13-auth-jwt`	Samuel Correa	N. Espinosa, G. Mejía	**Merged**
#5	`feature/HU-tests-qa`	Gustavo Mejía	N. Espinosa, S. Correa	**Merged**

Nota. Cada Pull Request fue sometido a revisión de pares (Code Review) y verificación de tests antes de su fusión a `develop`.

8.2 Versionamiento Semántico y Etiqueta Oficial

Al completar la consolidación y pruebas de la Entrega 2, se realizó la fusión hacia la rama `main` y se estampó el tag de versión oficial:

```
git tag -a v1.0.0-entrega2 -m "Release v1.0.0-Entrega2: Desarrollo y Pruebas (Grupo 4)"
```

Los mensajes de commit siguieron la especificación de *Conventional Commits* (ej. `feat(auth): ...`, `fix(repo): ...`, `test(concurrency): ...`, `docs: ...`), asegurando trazabilidad histórica completa.

9. Manual de Instalación, Configuración y Acceso Remoto

9.1 Instrucciones de Compilación y Ejecución Local (Zero Setup)

Para que el docente y el equipo puedan evaluar el proyecto de inmediato sin requerir configuración previa de motores de base de datos, el proyecto incluye el wrapper de Maven (`mvnw.cmd`) y un perfil predeterminado con base de datos H2 en memoria:

1. **Ejecutar la Suite de Pruebas Automatizadas:**

```
cd backend
.\mvnw.cmd test
```

2. **Iniciar el Servidor Spring Boot:**

```
.\mvnw.cmd spring-boot:run
```

   El servidor iniciará en el puerto `8080` en aproximadamente 4.5 segundos.

3. **Acceder a la Aplicación en el Navegador:**
   - Interfaz Web SPA: `http://localhost:8080`
   - Documentación Swagger UI: `http://localhost:8080/swagger-ui/index.html`
   - Consola H2 Database: `http://localhost:8080/h2-console` (JDBC: `jdbc:h2:mem:canchasdb`, Usuario: `sa`, Clave: vacía)

9.2 Enlace Público Seguro en Internet (Cloudflare Tunnel)

Para permitir la evaluación remota desde teléfonos celulares o computadores fuera de localhost, se habilitó un túnel público seguro con certificado SSL:

```
https://treasurer-exists-unknown-anime.trycloudflare.com
```

Este enlace permite acceder a la aplicación web, Swagger y la API REST desde cualquier conexión a Internet del mundo mientras el túnel se encuentre activo.

9.3 Despliegue Opcional con Docker Compose (MySQL 8 y phpMyAdmin)

Si se desea evaluar el sistema contra un motor relacional de producción, la raíz incluye `docker-compose.yml`:

```
docker-compose up -d
.\mvnw.cmd spring-boot:run -Dspring-boot.run.profiles=mysql
```

10. Verificación de Cumplimiento de Criterios de Evaluación

La Tabla 6 sintetiza la auditoría de cumplimiento contra cada uno de los criterios formales establecidos en la rúbrica oficial de la Entrega 2:

Tabla 6

Matriz de cumplimiento frente a la rúbrica de evaluación de la Entrega 2

Criterio de Evaluación	Evidencia Técnica y Justificación de Ingeniería	Grado de Logro
Implementación Funcional Básica	CRUD completo de escenarios y franjas de 60 min (HU-01 a HU-04), creación de reservas usuario-franja-fecha (HU-05), bloqueo de fechas pasadas (HU-08), cancelación inmediata (HU-06) y mantenimiento preventivo (HU-21).	**Excelente (100%)**
Manejo de Concurrencia y Anti-Overbooking	Aislamiento `REPEATABLE_READ`, bloqueo pesimista en base de datos (`PESSIMISTIC_WRITE`) y prueba multihilo con 8 hilos simultáneos demostrando cero colisiones horarias.	**Excelente (100%)**
Seguridad y Autenticación	Autenticación sin estado con tokens JWT (RFC 7519), contraseñas cifradas en BCrypt cost 12 (RNF-03), roles RBAC y aislamiento de reservas por usuario (HU-19).	**Excelente (100%)**
Pruebas Automatizadas Unitarias y QA	Suite de 23 pruebas ejecutadas con JUnit 5, Mockito y MockMvc con 100% de tasa de éxito (0 fallos). Colección oficial de Postman v2.1.0 con 18 casos probados.	**Excelente (100%)**
Interfaz Web de Usuario (SPA)	Implementación reactiva de wireframes WF-01 a WF-04 en HTML5/CSS3/JS con catálogo multideporte, matriz de colores (verde/rojo/azul) y autogestión de reservas.	**Excelente (100%)**
Control de Versiones y GitFlow	Ramas `main`, `develop`, `feature/*`, Pull Requests con revisión cruzada de código, commits convencionales y tag semántico `v1.0.0-entrega2`.	**Excelente (100%)**
Documentación Técnica Formal	Informe maestro estructurado bajo Normas APA 7ma Edición, Manual de Instalación, Guía de Arquitectura, Reporte de Pruebas y Colección de Postman.	**Excelente (100%)**

Nota. Evaluación auditada por el Coordinador de Calidad del equipo de trabajo.

11. Conclusiones y Planificación Hacia la Entrega 3

11.1 Conclusiones Técnicas de la Entrega 2

1. **Solidez del Núcleo Transaccional:** La combinación de transacciones con aislamiento repetible y bloqueo pesimista en base de datos erradica de forma definitiva el riesgo de *overbooking*, garantizando la integridad de datos bajo cargas concurrentes.
2. **Arquitectura Limpia y Extensible:** La adopción de Clean Architecture en Spring Boot 3 permite que cada componente (seguridad, servicios, persistencia, controladores) evolucione de manera independiente sin acoplamiento.
3. **Rigor en Aseguramiento de Calidad:** La batería de 23 pruebas automatizadas y la colección de Postman demuestran que el sistema no solo funciona en casos ideales, sino que responde de manera segura ante casos de borde (colisiones, fechas pasadas, cancelaciones no autorizadas).
4. **Experiencia de Usuario Fluida:** La implementación reactiva de los wireframes proporciona una experiencia ágil tanto para el deportista que aparta una cancha como para el administrador que gestiona el complejo.

11.2 Planificación y Hoja de Ruta para la Entrega 3 (Sprints 4 y 5)

Con el núcleo de reservas consolidado, el esfuerzo del equipo en los Sprints 4 y 5 se concentrará en las siguientes historias de usuario:

- **Módulo de Analítica y Dashboard (HU-09, HU-10):** Incorporación de gráficos interactivos con Chart.js para visualizar histogramas de horas pico nocturnas y porcentaje de ocupación mensual por escenario deportivo.
- **Exportación Tabular a CSV (HU-11, HU-12):** Generación de reportes de auditoría descargables filtrados por rango de fechas para la administración deportiva.
- **Pruebas de Aceptación de Usuario (UAT):** Validación de usabilidad con deportistas y personal administrativo de la universidad.
- **Despliegue Productivo en la Nube:** Publicación continua en plataforma cloud (Render/Railway) y preparación de la sustentación presencial final.

Referencias Bibliográficas

American Psychological Association. (2020). *Publication manual of the American Psychological Association* (7th ed.). https://doi.org/10.1037/0000165-000

Beck, K. (2003). *Test-driven development: By example*. Addison-Wesley Professional.

Cohn, M. (2005). *Agile estimating and planning*. Prentice Hall PTR.

Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* (Doctoral dissertation). University of California, Irvine.

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.

IEEE Computer Society. (1998). *IEEE Std 830-1998: IEEE Recommended Practice for Software Requirements Specifications*. IEEE. https://doi.org/10.1109/IEEESTD.1998.88286

International Organization for Standardization. (2011). *Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models* (ISO/IEC Standard No. 25010:2011). ISO.

Martin, R. C. (2018). *Clean architecture: A craftsman's guide to software structure and design*. Prentice Hall.

Nottingham, M., Wilde, E., & Dalal, S. (2023). *Problem Details for HTTP APIs* (RFC 7807bis). Internet Engineering Task Force. https://datatracker.ietf.org/doc/html/rfc7807

Pressman, R. S., & Maxim, B. R. (2020). *Software engineering: A practitioner's approach* (9th ed.). McGraw-Hill Education.

Schwaber, K., & Sutherland, J. (2020). *The Scrum Guide: The Definitive Guide to Scrum: The Rules of the Game*. Scrum.org.

Walls, C. (2022). *Spring in Action* (6th ed.). Manning Publications.